



University of Massachusetts Amherst

Department of Microbiology¹

Department of Electrical and Computer Engineering²

Mycelial Path Viewer & Export Tool

Complete Software Documentation

Version 1.0 · May 2026

Prepared by: Cooper Richman²

Supervisors

Dr. Sang Hyun Lee¹

Dr. Bill Leonard²

Contents

1	Introduction	1
1.1	Key Capabilities	1
1.2	Primary Exported Metrics	2
2	Software Overview	3
2.1	Control Overview	3
2.2	Junction Review (Phase 1)	4
2.2.1	Phase 1 Workflow	4
2.2.2	Junction State Reference	5
2.3	Analysis Mode (Phase 2)	5
2.3.1	Phase 2 Features	6
3	Installation and Setup	7
4	Input Data Requirements	9
5	Typical Workflow & Pipeline	10
6	Explanation of Outputs	12
6.1	XXX_frame_summary.csv	12
6.2	XXX_full_paths.csv	13
6.3	XXX_tip_data.csv	14
7	Adjustable Parameters	16
7.1	Physical Calibration	16
7.2	Anchor / Origin Detection	16
7.3	Junction ID Tracking	17
7.4	Mouse Interaction Thresholds	17

7.5	Display Defaults	17
7.6	Pairing Behavior	18
8	Limitations & Known Issues	19
9	Complete Example	21
10	Future Improvements	27
11	Appendix	29
11.1	Algorithm Summary	29
11.2	Faux Tip Mechanism	30
11.3	Colour Reference	31
11.4	Change Log	31

Chapter 1

Introduction

The **Mycelial Path Viewer & Export Tool** is an advanced Python-based analytical suite designed for the topological quantification of evolving branched mycelial networks. Utilizing a geodesic skeleton-based graph approach, the software enables users to extract high-fidelity growth data from time-lapse microscopy of filamentous structures.

The tool bridges the gap between raw image processing and formal graph theory by mapping physical skeleton pixels to a dynamic coordinate system. Pre-processed `.tiff` image stacks are ingested and processed frame-by-frame. Detected structural features such as, tips, junctions, and the network origin, are resolved into a queryable graph using Dijkstra shortest path routing, and every measurement is exportable to structured CSV files for downstream analysis.

Within this codebase are both executables (`.exe` files) and the original Python (`.py`) files. `Mycelial Path Viewer & Export Tool.exe` was created from `Mycelial Path Viewer Export Tool.py`, and `Regraphing Fungal Growth Viewer.exe` was created from `Regraphing Growth.py`. The source code for the `.exe` files can no longer be altered, i.e. their functionality can no longer be changed. However, `.py` files can be freely edited to fix bugs and add more functionality.

1.1 Key Capabilities

- Detection of branch **tips**, **junctions**, **crossovers**, and the network **origin** in each frame.
- Automatic, frame-to-frame **persistent junction ID tracking** using config-

urable radius proximity matching.

- **Dijkstra pathfinding** that pairs each tip to its nearest available junction.
- Interactive **junction state review** (normal, crossover $\blacklozenge$, parallel crossover $\blacksquare$, removed $\times$) with automatic forward propagation across frames.
- Manual **path override** capability: the user may re-route any tip to any junction or to the network origin.
- **Faux (ghost) tip** support for scenarios where two branches overlap at the same pixel location.
- Export of three complementary CSV files covering per-tip measurements, full pixel-level path data, and per-frame aggregate statistics.
- A companion regraphing script that replots each hypha individually per frame alongside general growth information.

1.2 Primary Exported Metrics

Metric	Unit(s)	CSV file
Path length	px / μm	<code>_tip_data</code>
Growth speed per frame	px/frame / $\mu\text{m}/\text{frame}$	<code>_tip_data</code>
Growth direction	degrees	<code>_tip_data</code>
Number of tips per frame	count	<code>_frame_summary</code>
Number of junctions (by type)	count	<code>_frame_summary</code>
Skeleton area	px^2 / μm^2	<code>_frame_summary</code>
Geodesic pixel count	count	<code>_frame_summary</code>
Untraced pixel count	count	<code>_frame_summary</code>
Full per-pixel path data	px / μm coords	<code>_full_paths</code>

Chapter 2

Software Overview

The tool is a single Python script (`Mycelial Path Viewer Export Tool.py`) driven by a Tkinter GUI window with an embedded Matplotlib canvas. The workflow is divided into two sequential phases:

- (i) **Phase 1 — Junction Review:** the user inspects and curates detected junctions before any path computation occurs.
- (ii) **Phase 2 — Analysis:** tip-to-junction pairings are computed, visualized, and can be exported to CSV.

2.1 Control Overview

The controls are designed for keyboard-driven navigation alongside point-and-click interaction. Phase applicability is noted in the rightmost column.

Control	Functionality	Phase
Mouse Scroll Wheel	Zoom in/out from the current cursor position	Both
Middle Mouse + Drag	Pan the image	Both
Shift + Left Drag	Pan the image (alternative)	Both
Left / Right Arrow Key	Advance the frame forward or backward	Both
Ctrl + C	Copy current view to clipboard	Both
Esc	Cancel the current active mode	Both
Left Click on junction	Cycle junction state: normal $\rightarrow$ cross $\times$ $\rightarrow$ cross $\parallel$ $\rightarrow$ removed $\rightarrow$ normal $\rightarrow$...	Both
O	Toggle Override Path mode	Analysis
F	Toggle Faux Tip mode (see §11.2)	Analysis
R	Refresh pairings for the current frame	Analysis
Right Click on tip	Rename tip label or cancel active mode	Analysis

Note on the F (Faux Tip) shortcut: While implemented, faux tip functionality does not work completely as intended in the current release. It should not cause crashes or corrupt existing data, but will not act as intended. See Chapter 8 for details.

2.2 Junction Review (Phase 1)

Phase 1 is the first screen the user sees after launching the tool and selecting a file. The large canvas on the left displays the skeleton for the current frame. Junction zones are rendered as colored blobs with a centroid dot and a persistent ID number label. Above the growth canvas, a status bar displays quick summary information for the current frame.

2.2.1 Phase 1 Workflow

- Navigate through all frames using the Left/Right arrow keys or the Prev/Next buttons.
- **Left-click any junction** to cycle its state: Normal (purple circle) $\rightarrow$ Crossover

◆ (orange diamond) → Parallel Crossover ■ (orange square) → Removed × (grey X) → back to Normal.

- Crossover ◆ (orange diamond): Used when two hyphae meet and cross one another, continuing in their respective directions.
- Parallel Crossover ■ (orange square): Used when two hyphae overlap for a few frames, but then continue growing in directions where they do not properly cross over one another.
- A state change propagates **forward automatically** to all subsequent frames sharing the same global junction ID. This avoids having to re-label the same physical junction in every frame.
- **Hover** over a junction centroid to see a distance tooltip listing how far neighboring junctions are (useful for verifying the match radius).
- Adjust the **Match Radius slider** to control how far apart two junctions can be (in pixels) and still be considered the same junction across consecutive frames.
- When satisfied, click ‘**Finalize & Track ►**’ to run the ID tracker and compute all pairings, transitioning to Phase 2.

2.2.2 Junction State Reference

State	Marker	Color	Effect on Pairing
Normal	●	Purple #e040fb	Included in pairing
Crossover (×)	◆	Orange #ff5e00	Excluded from pairing
Parallel Crossover ()	■	Orange #ff5e00	Excluded from pairing
Removed	×	Dark grey #555555	Fully ignored

2.3 Analysis Mode (Phase 2)

After clicking ‘Finalize & Track’, the tool transitions to Phase 2. The same canvas now displays colored pairing paths connecting each tip to its assigned junction, star markers at each tip, and ID badge labels.

2.3.1 Phase 2 Features

- **Hover over a path** to highlight it and display its length (in px and μm) in the title bar.
- **Override Path (O)**: Click a tip to select it (it glows green), then click a junction or the origin marker $+$ to manually reroute the pairing. Manually overridden paths are drawn dashed.
- **Faux Tip (F)**: Creates a ghost copy of a real tip at the same pixel. The faux tip participates in the greedy algorithm independently and is exported with `is_faux = 1` in all CSV files. Please note that this functionality is not currently working as intended.
- **Origin capacity** ($+/-$ buttons): controls how many tips are allowed to pair to the network origin (increases/decreases by 1 at a time).
- **Imbalance warning (Δ)**: shown in the title bar when the count of tips does not equal the count of active (normal) junctions.
- **Re-track button**: re-runs junction ID assignment with the current match radius and flashes a confirmation banner.
- **Export CSV**: saves all three output files (see Chapter 6).
- **Back to Review**: returns to Phase 1 with all junction states and manual overrides preserved.

Chapter 3

Installation and Setup

The tool is a self-contained Python script. It requires **Python 3.8 or later** and the following third-party packages.

You will also most likely want some sort of Integrated Development Environment (IDE) to help view and edit the code. A common and free option is Visual Studio Code (VSCode). There are online installation guides for how to install it. Additionally, whatever machine that runs the code will need to have Python and the following dependencies installed. Again, there are installation guides and webpages on how to do this as well. All of the code for this project can be found on the GitHub. GitHub can also be understood via online tutorials.

Package	Install command	Role
numpy	<code>pip install numpy</code>	Array/matrix mathematics and image representation
matplotlib	<code>pip install matplotlib</code>	Rendering the GUI canvas and all visual overlays
scikit-image	<code>pip install scikit-image</code>	Reading <code>.tiff/.tif</code> image stacks
scipy	<code>pip install scipy</code>	Convolution (neighbor counting) and binary dilation
tkinter	(bundled with Python)	GUI window framework
pywin32	<code>pip install pywin32</code>	Windows clipboard copy (optional)
Pillow	<code>pip install Pillow</code>	Image conversion for clipboard (optional)

Quick install of all required packages:

```
pip install numpy matplotlib scikit-image scipy
```

To run the script:

```
python Mycelial Path Viewer Export Tool.py
```

On launch, a file-picker dialog appears immediately. Select a skeletonized `.tiff` or `.tif` stack. The tool processes the file and opens the main GUI window, printing per-frame statistics (frame count, anchor position, tip and zone counts) to the terminal.

The included `.exe` files have all dependencies wrapped in them, but to run the source code these will need to be installed.

Platform note: The clipboard-copy feature (Ctrl+C) requires `pywin32` and `Pillow` on Windows. If these packages are not found, the current view is instead saved as a PNG file (`_clipboard_export.png`) next to the script.

Chapter 4

Input Data Requirements

Input data format is critical. The script is designed to accept **skeletonized binary TIFF stacks** representing a time series of mycelial fungal growth.

- **File format:** `.tiff` or `.tif` (both extensions accepted).
- **Stack type:** Multi-frame (3-D array: frames $\times$ rows $\times$ cols).
- **Pixel values:** Binary (any non-zero value is interpreted as a skeleton pixel; zero is background). The script binarizes internally with `raw = raw > 0`.
- **Skeleton quality:** Images must already be skeletonized (1-pixel-wide skeleton lines). The tool does ***not*** perform skeletonization itself.
- **Pixel size:** The physical size of each pixel in micrometres is set via the `PIXEL_SIZE_UM` constant in the script (default: **1.73 $\mu\text{m}/\text{px}$**). This must be calibrated correctly before running to obtain accurate μm measurements in the CSV exports.
- **Network topology:** The tool assumes a single connected mycelial network with one identifiable stable origin (root) pixel. Completely disconnected skeletons in a frame will still process, but some tips may remain unpaired.

Pre-processing recommendation: Apply skeletonization (e.g., morphological skeletonization via `skimage.morphology.skeletonize` or FIJI) before passing them to this tool. See Chapter 9 for a complete example.

Chapter 5

Typical Workflow & Pipeline

The following seven steps describe a complete session from raw images to exported data. See Chapter 9 for a complete example from `.nd2` to the resulting `.csv`'s.

- (i) **Prepare input.** Pre-process data via something like FIJI that allows for cropping, thresholding, binarization, and skeletonization. Export this skeletonized image stack as a `.tiff` stack. Open `Mycelial Path Viewer Export Tool.py` script in something such as VSCode to edit the necessary parameters such as `PIXEL_SIZE_UM` to match your microscope calibration.
- (ii) **Launch & select file.** Run the script. A file picker dialog opens. Select your `.tiff` stack. The tool loads all frames, detects the global anchor/origin, and builds per-frame data structures.
- (iii) **Phase 1: Junction Review.** The GUI opens in Phase 1. Navigate all frames. Left-click junctions to mark crossovers or remove spurious or erroneous junctions. Adjust the match radius slider so that the same physical junction is consistently linked across frames. Use the hover tooltip to verify inter-junction distances.
- (iv) **Finalize & Track.** Click 'Finalize & Track ►'. The junction ID tracker runs across all frames and the greedy pairing algorithm assigns each tip to its nearest available junction. The tool transitions to Phase 2.
- (v) **Phase 2: Review Pairings.** Inspect the colored path overlays. Hover over any path to see its length. Use Override Path (O) to manually reroute any incorrect pairing. Add Faux Tips (F) if two branches overlap at a single pixel.

- (vi) **Export CSV.** Click ‘Export CSV...’. Select a base filename. Three CSV files are written automatically (see Chapter 6). You can return to Phase 1 at any time to adjust junction states without losing overrides.
- (vii) **Downstream analysis.** Import the CSV files into your analysis environment (Python, R, Excel, etc.). Use the companion regraphing script to visualize each hypha’s growth trajectory individually per frame as well as some other information about the growth.

Chapter 6

Explanation of Outputs

When ‘Export CSV...’ is clicked, a single save dialog appears. The filename the user provides becomes the prefix for **three automatically named CSV files**. For example, entering `mycelium_test` creates:

Output file	Contents summary
<code>mycelium_test_frame_summary.csv</code>	One row per frame, aggregate statistics
<code>mycelium_test_full_paths.csv</code>	One row per skeleton pixel per path per frame
<code>mycelium_test_tip_data.csv</code>	One row per tip per frame, measurements & speed

The outputs are split into three files to ensure no single file becomes unmanageably large.

6.1 XXX_frame_summary.csv

One row per frame. Contains aggregate per-frame statistics.

Column	Type	Description
<code>frame</code>	int	Frame index, zero-based
<code>n_tips</code>	int	Number of detected real tips in this frame
<code>n_faux_tips</code>	int	Number of faux (ghost) tips active in this frame
<code>n_junctions_normal</code>	int	Count of normal (active) junctions
<code>n_junctions_crossover</code>	int	Count of crossover or parallel crossover junctions
<code>n_junctions_removed</code>	int	Count of removed junctions
<code>skeleton_area_px2</code>	int	Total skeleton pixel count (area in px ²)
<code>skeleton_area_um2</code>	float	Skeleton area in μm^2 (<code>PIXEL_SIZE_UM</code> ² × px ²)
<code>n_geodesic_pixels</code>	int	Total pixels covered by all pairing paths
<code>n_untraced_pixels</code>	int	Skeleton pixels not on any pairing path (loops, spurs, disconnected strands)

6.2 XXX_full_paths.csv

One row per skeleton pixel per pairing path, per frame. This is the most granular output. Includes both **geodesic** (path-traced) pixels and **untraced** pixels, or skeleton pixels that belong to loops, spurs, or disconnected strands not covered by any tip-to-junction path.

Column	Type	Description
<code>tip_id</code>	string	Label of the tip this path belongs to (e.g. J3, orig1, faux.2). Empty for untraced pixels.
<code>frame</code>	int	Frame index (zero-based)
<code>step_index</code>	int	Position of this pixel along the path (0 = tip). -1 for untraced pixels.
<code>pixel_x</code>	int	Column (x) coordinate of this pixel in image space
<code>pixel_y</code>	int	Row (y) coordinate of this pixel in image space
<code>pixel_x_um</code>	float	Column coordinate converted to μm
<code>pixel_y_um</code>	float	Row coordinate converted to μm
<code>path_type</code>	string	'geodesic' for measured paths; 'untraced' for unused skeleton pixels
<code>is_manual_override</code>	0/1	1 if this path was set by the user via Override Path; 0 otherwise
<code>is_faux</code>	0/1	1 if this path belongs to a faux (ghost) tip; 0 otherwise

6.3 XXX_tip_data.csv

One row per tip per frame. This is the primary measurement file for growth analysis. Speed and direction are computed between consecutive frames for each labeled tip.

Column	Type	Description
tip_id	string	Consistent label across frames (e.g. J3, orig1, faux_2)
frame	int	Frame index (zero-based)
tip_x_px	int	Tip column (x) coordinate in pixels
tip_y_px	int	Tip row (y) coordinate in pixels
paired_junction	string	Local zone index, or 'origin' / 'unpaired'
junction_global_id	string	Persistent global junction ID. 'origin' for origin pairings.
path_length_px	float	Geodesic path length from tip to its junction, in pixels
path_length_um	float	Path length converted to μm
speed_px_per_frame	float	Euclidean tip displacement from previous frame, in pixels
speed_um_per_frame	float	Same displacement in μm
direction_deg	float	Growth direction in degrees (0° = right, counter-clockwise positive)
is_manual_override	0/1	1 if this pairing was set manually via Override Path
is_faux	0/1	1 if this tip is a faux (ghost) tip

Chapter 7

Adjustable Parameters

All tunable parameters are defined near the top of the script (Section 3 of the source code). They are grouped below by category. Parameters marked [**runtime**] can also be adjusted interactively via the GUI without restarting the script.

7.1 Physical Calibration

Parameter	Default	Description
PIXEL_SIZE_UM	1.73	Micrometres per pixel. Must match microscope calibration. Used for all μm conversions in CSV exports.

7.2 Anchor / Origin Detection

Parameter	Default	Description
ANCHOR_MIN_FRAME_FRACTION	0.50	Fraction of frames a pixel must be present in to be considered stable for anchor detection.
ANCHOR_SNAP_RADIUS	20 px	Search radius when snapping the computed anchor centroid to the nearest actual skeleton pixel.

7.3 Junction ID Tracking

Parameter	Default	Description
DEFAULT_MATCH_RADIUS	20 px	Default junction ID matching radius. Two junction centroids must be within this distance (pixels) to be considered the same junction across consecutive frames. Also adjustable at runtime via the Match Radius slider [runtime].

7.4 Mouse Interaction Thresholds

Parameter	Default	Description
HOVER_RADIUS_PX	6 px	Mouse must be within this distance of a path segment to register as hovering over it.
JUNCTION_HOVER_PX	14 px	Mouse must be within this distance of a junction centroid to register as hovering over it.
CLICK_ZONE_RADIUS_PX	15 px	Mouse must be within this distance of a junction centroid to register as a click.
CLICK_TIP_RADIUS_PX	10 px	Mouse must be within this distance of a tip pixel to register as a click.

7.5 Display Defaults

Parameter	Default	Description
invert_view	True	If True, the canvas renders with a white background (easier to see skeleton on screen). Set to False for a dark background.
show_overlay_master	True	Initial visibility state of the path/label overlay.

7.6 Pairing Behavior

Parameter	Default	Description
origin_capacity	1	Number of tips allowed to pair to the network origin. Adjustable at runtime via the $+/-$ buttons in the right panel [runtime].

Runtime adjustments: The **Match Radius** slider and **Dilation** slider (cosmetic skeleton line thickness, in pixels) can be adjusted at any time without restarting. After changing the match radius, click ‘ Re-track with this radius’ to apply the new value to junction ID assignment across all frames.

Chapter 8

Limitations & Known Issues

The following limitations and known issues are present in the current version of the tool. They are documented here for transparency and to guide future development (see also Chapter 10).

Faux Tips (F) — Partial Implementation

The faux tip feature is implemented but does not work completely as intended. Faux tips appear correctly as triangle markers, but their functionality in terms of pathing is extremely limited. The feature should not cause crashes or corrupt existing data, but results that use them should be verified manually.

Retracing After Override

After a manual path override, the automatic re-pairing of non-overridden tips works correctly in most cases. However, in complex topologies with many simultaneous overrides, the re-routing of remaining tips may not fully converge to the optimal assignment in a single pass. All paths should be inspected after an override to ensure that this is handled.

.tif Stack Issues with Binary Inversion

It has been observed that with certain skeletonized .tif stacks the script can misinterpret what is the background versus what are the skeleton pixels. This causes processing time to be immense, as the script thinks that the skeleton is massive. It is most likely that this is due to the binary data of the .tif stack is reversed i.e. all of the background pixels are labeled as 1 and all of the skeleton pixels are labeled as 0 when it should be vice versa. This should be able to be remedied by using tools such as FIJI to invert the data.

Parallel Crossover Type (■): Limited Distinction

The parallel crossover state is implemented in the UI and exported to the CSV, but its functional distinction from a regular crossover (◆) is limited. Both states exclude the junction from the greedy pairing algorithm equally. To use this feature more faithfully, changes would need to be made to ensure it re-routes the tip paths correctly.

Single Connected Network Assumption

Anchor and origin detection assume a single mycelial network with pixels present in at least 50% of frames. Highly fragmented skeletons or networks that appear in fewer frames may result in a suboptimal anchor placement.

Windows-Only Clipboard Copy

The Ctrl+C clipboard copy uses `win32clipboard`, which is Windows-specific. On Linux and macOS, the current view is instead saved as a PNG file. A cross-platform could be beneficial.

No Undo / Redo

There is currently no undo/redo functionality. Resetting zone states or clearing overrides is permanent within a session (though returning to Phase 1 preserves junction states between phases).

Performance on Large Stacks

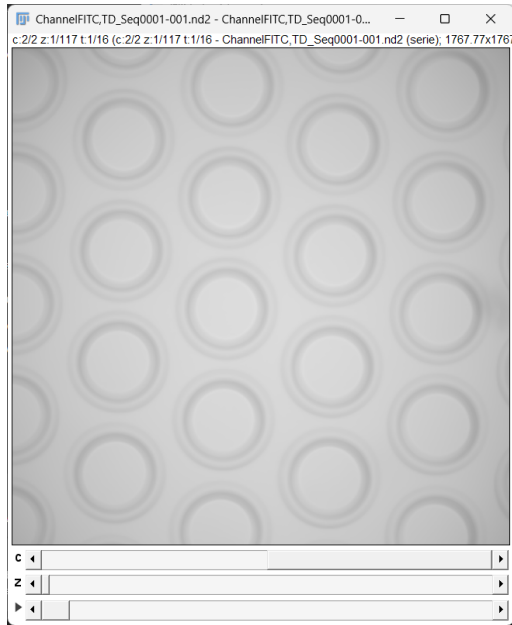
Stacks with many frames (> 50) and/or large images ($> 1000 \times 1000$ px) may experience slow initial loading and per-frame render times, as the full skeleton graph is rebuilt for each frame and Dijkstra's algorithm is run independently for every tip.

Chapter 9

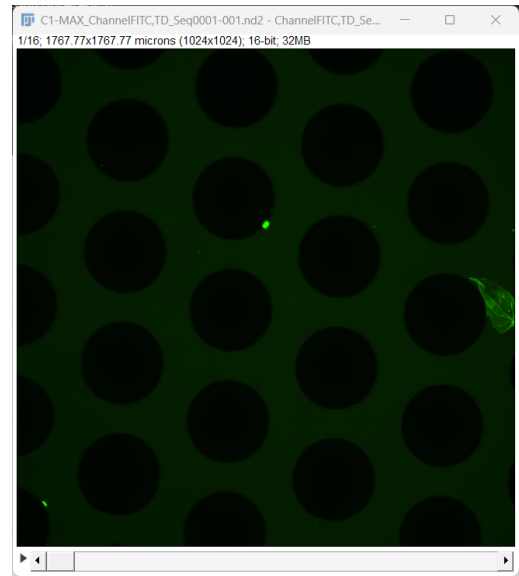
Complete Example

The following walkthrough describes a complete session using a recorded `.nd2` file from an experiment conducted in the UMass Amherst Biofluids Lab. This will go from the raw `.nd2` file to the exported metrics and analysis script. **Note that all of the preprocessing will be done using FIJI.**

- (i) **Prepare the `.nd2`.** Typically the `.nd2` file will come in a way that must be further processed before being entered into the script. Note that the desired input for the script is a single-pixel wide skeleton `.tiff` stack with its only dimension being time (or frames). There are many ways to arrive at this point from an `.nd2` file, the following procedure is just one of them.
 - (a) Open FIJI
 - (b) Select File > Open
 - (c) Choose the desired `.nd2` file
 - (d) Select the view intended for analysis
 - (e) Do a Max-Z projection by selecting Image > Stacks > Z Project and setting the **Projection Type** to **Max Intensity**
 - (f) Split the channels by selecting Image > Color > Split Channels, and focus on the fluorescent channel



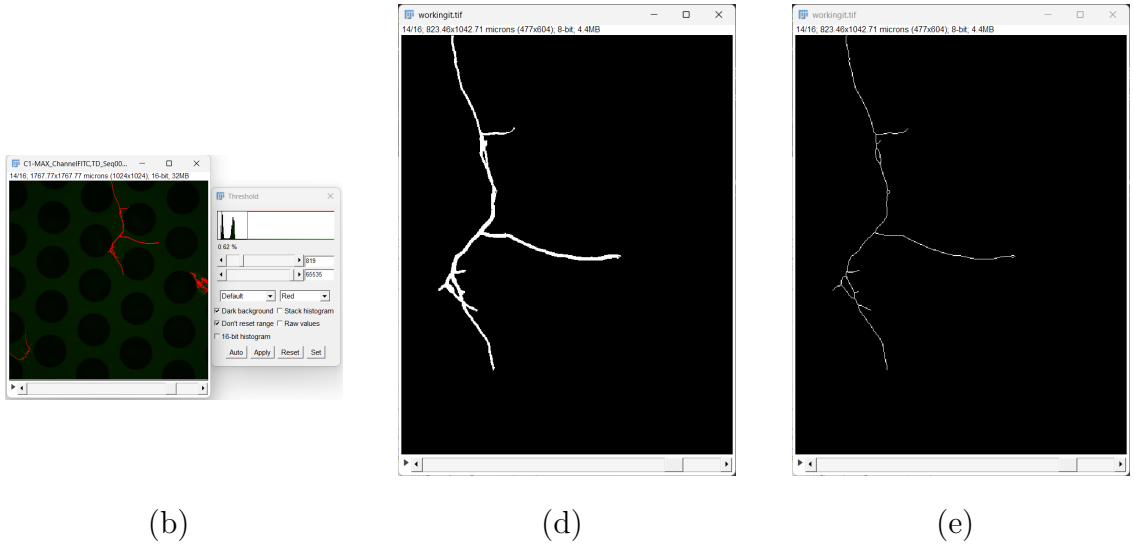
(d)



(f)

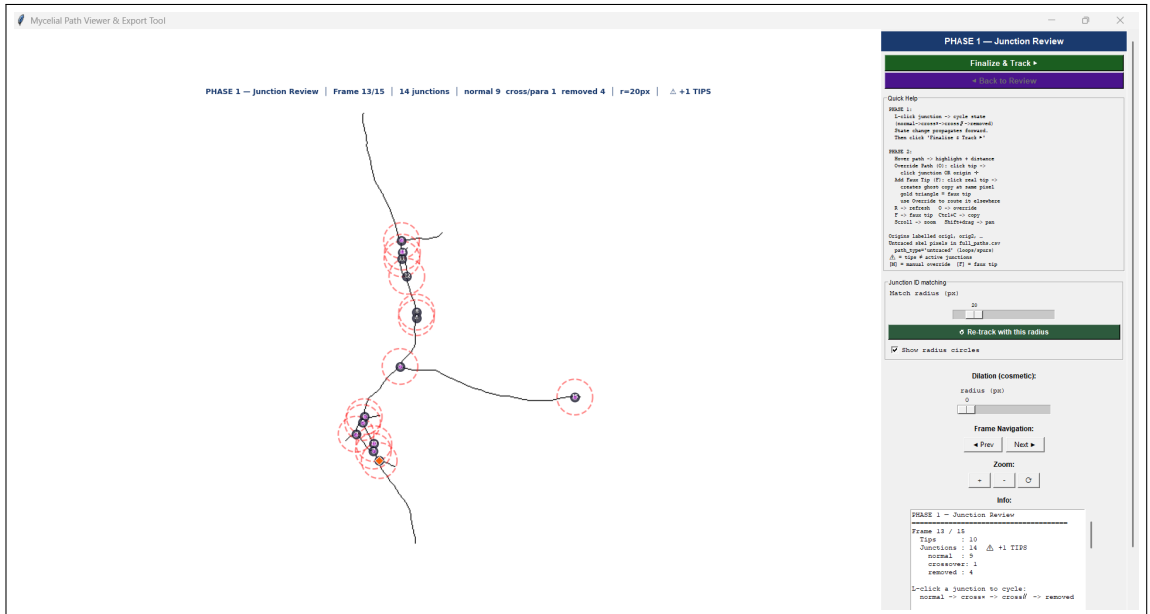
(ii) **Threshold, Binarize, Skeletonize.** This must now be converted into a skeleton suitable to be input for the script. Note that a different frame (frame 14) of the `.nd2` will be shown here to help demonstrate the steps more clearly.

- (a) Select Image > Adjust > Threshold
- (b) Move the **top** slider to the **right** until only the the growth is highlighted (leave the rest of the settings default) and press **Apply**
- (c) Uncheck **Calculate threshold for each image** and press **Okay**
- (d) It is now advised to crop the view to be more focused on the growth and remove small disconnected noise (there are several ways to do this, one approach could be Edit > Invert, selecting all spurious outliers with the rectangle tool and pressing **del** for each frame, and then inverting back)
- (e) Skeletonize the growth by selecting Process > Binary > Skeletonize
- (f) Save the skeletonized `.tiff` stack by selecting File > Save As



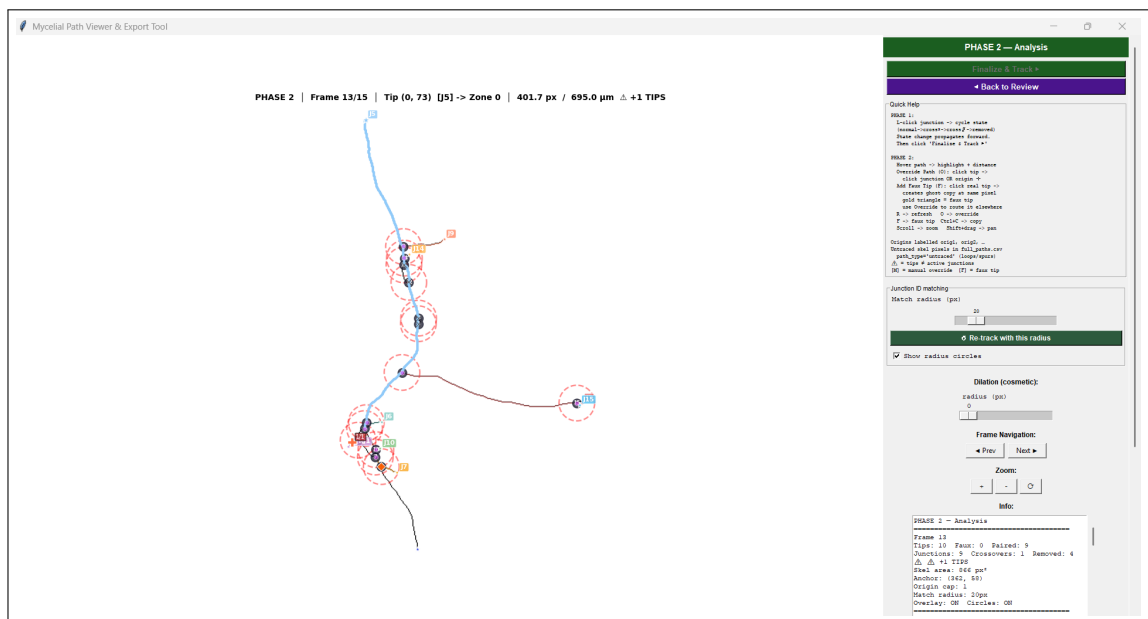
(iii) **Launch the Script.** Run `python Final_Tracking_Version.py`. Select the growth that was just saved. The terminal prints the frame count, per-frame anchor positions, and per-frame tip and zone counts.

(iv) **Phase 1: Review Junctions.** The GUI opens on frame 0. Go through each frame and tidy up junctions that should be marked as crossovers or removed. Ensure that junction IDs are consistent across frames, and adjust the radius if necessary with the slider.



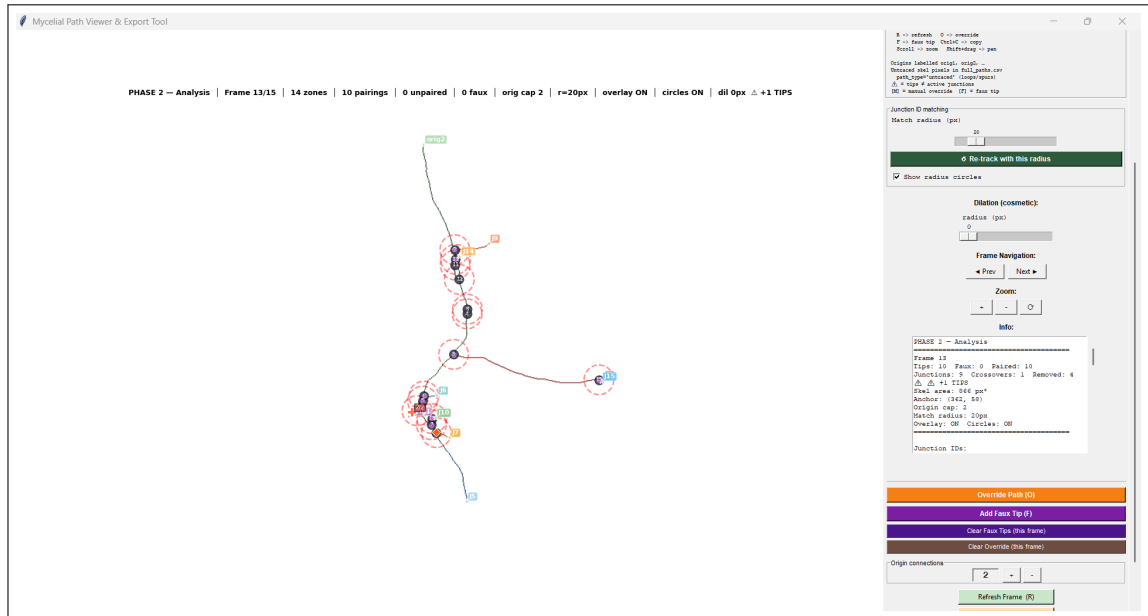
Frame 14/16 (shown as 13/15 due to 0-based indexing) during Phase 1.

(v) **Phase 2: Finalize.** Click 'Finalize & Track ►'. The algorithm assigns persistent IDs and computes pairings for all frames. The canvas switches to Phase 2 with colored path overlays.



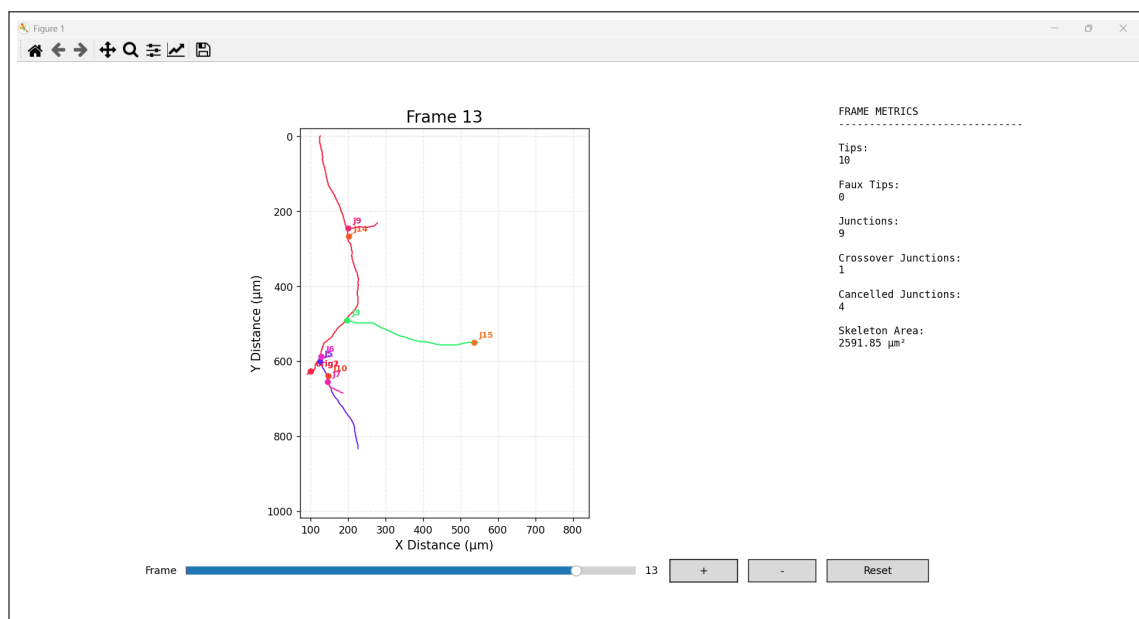
Initial pairings after pressing "Finalize & Track ►."

(vi) **Review Pairings.** After finalizing, verify the results before exporting. In this case it can be seen that the bottom-most tip was not paired to a junction (most likely due to too many tips and not enough junctions). This can be easily fixed by using the override function. Also pay attention to the amount of origin connections. By looking at early frames of this growth, it appears that there are two main branches that come from the origin, meaning that the amount of connections the origin should be able to handle is 2. This can also easily be changed by scrolling down and pressing the + button. Verify that after all changes the paths are where they are expected to be, and that none of them have been removed or altered by other operations.



Reviewed pairings by adding an origin connection and overriding a path.

- (vii) **Export.** Click ‘Export CSV...’ at the bottom of the scrollable right side. Navigate to the output folder and enter the filename. Three files are created: `XXX_frame_summary.csv`, `XXX_full_paths.csv`, and `XXX_tip_data.csv`.
- (viii) **Downstream Analysis.** Many analyses can be conducted using these .CSV files. By using Python, MATLAB, and similar tools, meaningful insights can be extracted from the data. One such tool comes with this code, a tool that takes all three .CSV files as an input, and regraphs each frame as well as shows general information about the growth on the right side. This is helpful for quick analysis, as well as seeing if the tip IDs have been consistent across frames, and what tips or junctions might have been missed during pairing and finalizing.



Frame 14 (again labeled as 13/15) with all of its hyphae mapped.

Chapter 10

Future Improvements

The following improvements are planned or under consideration for future versions of the tool.

- **Full faux tip implementation:** complete automatic pairing, frame-to-frame faux tip persistence, and robust edge-case handling for when a faux tip becomes a real tip.
- **Retrace after override:** after a manual override, re-trace all remaining paths with full edge exclusion to guarantee no skeleton segment is shared between any two tips.
- **True parallel crossover support:** graph-level distinction between a regular crossover and a parallel crossover, enabling correct routing through physically overlapping parallel branches.
- **Undo / Redo:** a session-level undo/redo stack for junction state changes and manual overrides.
- **Cross-platform clipboard:** replacement of `win32clipboard` with a cross-platform solution for clipboard support on macOS and Linux.
- **Performance optimization:** lazy graph construction (build only when a frame is first accessed) and caching of Dijkstra results to speed up navigation of large stacks.
- **Automatic crossover detection:** heuristic or machine-learning-based detection of likely crossover junctions to reduce manual review time in Phase 1.

- **Companion script documentation:** full documentation for the companion regraphing script that plots each hypha individually per frame alongside summary growth statistics.

Chapter 11

Appendix

11.1 Algorithm Summary

The core pairing algorithm operates as follows for each frame.

- (i) Build a **weighted adjacency graph** from the binary skeleton image. Cardinal edges (up/down/left/right) have weight 1.0; diagonal edges have weight $\sqrt{2} \approx 1.414$.
- (ii) Detect all **tip pixels** (pixels with exactly one skeleton neighbor, degree = 1) via a 3×3 convolution neighbor count.
- (iii) Detect all **junction zones** (pixels with three or more skeleton neighbors, degree ≥ 3), flood-filled into spatial clusters.
- (iv) **Snap the anchor** to the nearest skeleton pixel for this frame using a “sticky anchor” strategy: the anchor pixel from the previous frame is preferred if it is still on the skeleton, otherwise the nearest skeleton pixel within `ANCHOR_SNAP_RADIUS` is used.
- (v) **Pre-sort tips** by their unconstrained nearest-junction distance (Dijkstra with no edge exclusions). The tip geometrically closest to any junction goes first and gets first pick.
- (vi) **Greedy assignment:** for each tip in sorted order, run Dijkstra with the current `excluded_edges` set. Claim the nearest available normal junction. Mark all edges of the chosen path as excluded so subsequent tips cannot reuse the same skeleton segments.

- (vii) If **manual overrides** are present, run the algorithm only on non-overridden tips. The override paths' edges are pre-seeded into `excluded_edges` and the overridden junctions are excluded from the algorithm's candidate pool, preventing double-claiming.
- (viii) **Persistent junction IDs** are assigned across frames by greedy centroid proximity matching: for each new frame, each junction centroid is matched to the closest centroid from the previous frame within the match radius (closest pair assigned first, then excluded from further matching).

11.2 Faux Tip Mechanism

A **faux tip** (ghost tip) is a synthetic tip that occupies the same pixel location as a real tip but carries a completely separate identity (e.g., `faux_3`). It participates in the pairing algorithm independently, so a second path can depart from the same pixel to a different junction. This is useful when two branches overlap for several frames and only one tip is visible.

Faux tips are rendered as triangle markers. In the CSV exports they appear as separate rows with `is_faux = 1` and `tip_id` values prefixed `faux_`.

11.3 Colour Reference

Element	Color	Hex code
Normal junction blob	Purple	#e040fb
Crossover / parallel junction	Orange	#ff5e00
Removed junction	Dark grey	#555555
Real tip pixel	Blue	#384ce4
Faux tip pixel	Gold	#ffd600
Network origin	Red	#ff1744
Pairing paths	Cycling palette (10 colors)	#4fc3f7, #81c784, #ffb74d, ...
Manual override path (dashed)	Same as paired path	—
Selected tip (override mode)	Green	#00ff88
Junction hover highlight	White	#ffffff
Match-radius circle	Red	#ff0000

11.4 Change Log

- **June 1st, 2026:** Initial release. Polished and handed off to UMass Biofluids Lab. (C. Richman)

Bibliography

- [1] C. Baranger et al., “Microfluidic Monitoring of the Growth of Individual Hyphae in Confined Environments,” *Royal Society Open Science*, vol. 7, no. 8, p. 191535, Aug. 2020, doi: 10.1098/rsos.191535.
- [2] T. Chassereau et al., “Direct Evidence of Apex-Hypha Interactions During Vegetative Growth of Fungal Thallus via Comprehensive Network and Trajectory Extraction,” *arXiv preprint* arXiv:2507.06280, Apr. 2026, doi: 10.48550/arXiv.2507.06280.
- [3] Y. Chen et al., “CellTrack R-CNN: A Novel End-To-End Deep Neural Network for Cell Segmentation and Tracking in Microscopy Images,” *arXiv preprint* arXiv:2102.10377, Feb. 2021, doi: 10.48550/arXiv.2102.10377.
- [4] T. G. de Jong et al., “Fungal Tip Growth Arising through a Codimension-1 Global Bifurcation,” *arXiv preprint* arXiv:1909.05486, Sept. 2019, doi: 10.48550/arXiv.1909.05486.
- [5] E. Martínez-Galicia, A. F. Flores Enríquez, A. Puga, and B. Gutiérrez-Medina, “Analysis of the emerging physical network in young mycelia,” *Fungal Genetics and Biology*, vol. 168, p. 103823, 2023, doi: 10.1016/j.fgb.2023.103823.
- [6] R. Ren et al., “Deep Learning Application to Hyphae and Spores Identification in Fungal Fluorescence Images,” *Scientific Reports*, vol. 15, no. 1, p. 27222, Jul. 2025, doi: 10.1038/s41598-025-11228-y.
- [7] G. Sánchez-Orellana, S. Casas-Flores, and B. Gutiérrez-Medina, “Automated, continuous video microscopy tracking of hyphal growth,” *Fungal Genetics and Biology*, vol. 123, pp. 25–32, 2019, doi: 10.1016/j.fgb.2018.11.006.
- [8] P. Saponaro et al., “Three-Dimensional Segmentation of Vesicular Networks

of Fungal Hyphae in Macroscopic Microscopy Image Stacks,” *arXiv preprint* arXiv:1704.02356, Apr. 2017, doi: 10.48550/arXiv.1704.02356.

- [9] G. Steinberg et al., “Cell Biology of Hyphal Growth,” *Microbiology Spectrum*, vol. 5, no. 2, 2016, doi: 10.1128/microbiolspec.funk-0034-2016.
- [10] G. Vidal-Diez de Ulzurrun et al., “Fungal Feature Tracker (FFT): A Tool for Quantitatively Characterizing the Morphology and Growth of Filamentous Fungi,” *PLOS Computational Biology*, vol. 15, no. 10, p. e1007428, 2019, doi: 10.1371/journal.pcbi.1007428.